CLAUDE.md — Trylo.pk Build Instructions

This file is read automatically by Claude Code at the start of every session in this repository. Keep it in the project root.

What this project is

Trylo.pk is a curated fashion e-commerce app for Pakistan — a more trustworthy, personalized alternative to Daraz. Swipe-based discovery → behavioral memory → virtual try-on, built on a two-gate seller trust system. Women's fashion is the primary focus; men's fashion exists alongside in the same app via a department toggle.

Before doing any planning, scaffolding, or coding work, read `Trylo_Technical_Specification.md` in this repo's root in full. It is the source of truth for scope, data model, user flow, and the recommendation logic. Do not re-derive product decisions from this file alone — this file only holds constraints and process; the spec holds the actual product.

A prior interactive prototype (single-file HTML/JS, illustrated placeholder garments, no backend) is included as `prototype_reference.html` if present. It is a UX/behavior reference only — do not extend or port it. Use it only to resolve ambiguity about exact intended interaction details (copy text, timing, animation behavior) when the spec doesn't spell it out.

Non-negotiable constraints

These override any instruction to move faster, simplify, or "just get something working":

1. **No scraped/internet-sourced product images, ever.** Product photos come only from real onboarded sellers who own the rights. If no real seller photos exist yet for a product, render an honest "photo coming soon" placeholder — never substitute a stock image or scraped image, even temporarily, even for a demo.
2. **No custom-built virtual try-on model.** Do not attempt to implement body-accurate try-on via image manipulation, "pixel scaling," or any from-scratch computer vision pipeline. The correct architecture is integrating a third-party try-on API (see spec Section 7). If asked to build try-on before that integration is wired up, build the UI/flow only and leave the actual render step clearly stubbed and labeled as not-yet-live — never fake a try-on result.
3. **Typo-tolerant, category-faithful search is mandatory, not a nice-to-have.** A search for "abaya" (or a close misspelling of it) must return abaya products, every time, with no fallback path that can return an unrelated category. This was a confirmed bug in the prototype phase — do not reintroduce it.
4. **Never let a no-signal recommendation state fall back to raw database/insertion order.** When there's no swipe/taste data yet, actively enforce variety (different sellers, different sub-styles) in whatever is shown. This was also a confirmed prototype bug — variety must be deliberately engineered, not assumed.
5. **Seller identity, rating, and sales count must always be visible to the buyer** at every touchpoint (swipe card, results, product detail). Never hide or omit this — it's a core trust differentiator for this product, not a cosmetic detail.

6. **Try-on UI must only appear on the product detail page, never during swiping/discovery.** This was an explicit, deliberate product decision — don't "fix" it by adding try-on earlier in the flow even if it seems convenient.
7. **The "maybe" swipe gesture (up) is a reduced-weight POSITIVE signal, not a negative one.** Get this right in the scoring logic — it was a confirmed bug risk during prototyping.

How to handle ambiguity

- If the spec answers it, follow the spec exactly, including exact UX sequencing (search → 3 benchmark swipes → 2 confirmation swipes → results → product detail → try-on).
- If the spec is silent and the decision is minor (styling detail, copy wording, minor UX polish), make a reasonable choice and proceed — note the assumption in your response, don't block on it.
- If the spec is silent and the decision affects architecture, cost, data model, or scope (e.g. choice of try-on API vendor, payment provider, hosting), stop and ask rather than guessing — these decisions are expensive to reverse and belong to the founder.
- If asked to cut one of the non-negotiable constraints above "just for now" or "just for the demo," don't silently comply — flag it back explicitly and explain the risk, then proceed only if the founder confirms in writing in the conversation.

Suggested build order

Follow Section 13 of the spec unless told otherwise: seller onboarding/verification → product listing flow → search/taxonomy → swipe discovery flow → recommendation engine → results/product detail → order-intent handoff → feedback instrumentation (build alongside, not after) → reputation system → packaging-proof photo step → try-on API integration → refund/payment tracker.

Tech stack (per spec Section 12 — confirm before deviating)

React (mobile-first web) · Node.js/TypeScript or Python backend · PostgreSQL · a dedicated search layer for fuzzy/typo-tolerant matching (Typesense/Meilisearch, or Postgres `pg_trgm`) · S3-compatible object storage for seller photos · third-party API for try-on, not in-house.

Tone for any explanations back to the founder

Ahmad is a non-technical founder. When explaining technical decisions or tradeoffs back to him, explain plainly — what it costs, what it risks, what the alternative would be — rather than assuming familiarity with the underlying technology.